```python
# 1. Import required libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, Flatten, Dense
import numpy as np

# 2. Load MNIST dataset
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()

# 3. Preprocess the images
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

# Add channel dimension
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# 4. Build CNN model using Sequential API
model = Sequential([
    Input(shape=(28, 28, 1)),

    Conv2D(
        filters=16,
        kernel_size=(3, 3),
        strides=(2, 2),
        padding='same',
        activation='relu'
    ),

    Conv2D(
        filters=32,
        kernel_size=(3, 3),
        strides=(1, 1),
        padding='valid',
        activation='relu'
    ),

    Flatten(),

    Dense(10, activation='softmax')
])

# 5. Display model architecture
model.summary()

# 6. Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 7. Train the AI model
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

# 8. Evaluate the model
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("\nModel Evaluation")
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy * 100, "%")

# 9. Make prediction
predictions = model.predict(x_test[:5])

print("\nSample Predictions:")
for i in range(5):
    predicted_digit = np.argmax(predictions[i])
    print("Predicted:", predicted_digit,
          "| Actual:", y_test[i])

# 10. Save the trained AI model
model.save("feature_shrinking_cnn.keras")

print("\nAI Model successfully trained and saved!")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step

Model: "sequential"

┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━
━━━━━┓
┃ Layer (type)                    ┃ Output Shape           ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━
━━━━━┩
│ conv2d (Conv2D)                 │ (None, 14, 14, 16)     │
160 │
├─────────────────────────────────┼────────────────────────┼──────────
─────┤
│ conv2d_1 (Conv2D)               │ (None, 12, 12, 32)     │
4,640 │
```

```
├─────────────────────────────────┼────────────────────────┼──────────
─────┤
│ flatten (Flatten)               │ (None, 4608)           │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
─────┤
│ dense (Dense)                   │ (None, 10)             │
46,090 │
└─────────────────────────────────┴────────────────────────┴──────────
─────┘

 Total params: 50,890 (198.79 KB)

 Trainable params: 50,890 (198.79 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 17s 19ms/step - accuracy: 0.9355 - loss:
0.2225 - val_accuracy: 0.9752 - val_loss: 0.0878
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 20s 19ms/step - accuracy: 0.9775 - loss:
0.0751 - val_accuracy: 0.9825 - val_loss: 0.0617
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 20s 19ms/step - accuracy: 0.9833 - loss:
0.0543 - val_accuracy: 0.9835 - val_loss: 0.0585
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 16s 19ms/step - accuracy: 0.9864 - loss:
0.0414 - val_accuracy: 0.9833 - val_loss: 0.0579
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 21s 20ms/step - accuracy: 0.9898 - loss:
0.0328 - val_accuracy: 0.9863 - val_loss: 0.0542
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9867 - loss:
0.0418

Model Evaluation
Test Loss: 0.041759577664847374
Test Accuracy: 98.66999983787537 %
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 80ms/step

Sample Predictions:
Predicted: 7 | Actual: 7
Predicted: 2 | Actual: 2
Predicted: 1 | Actual: 1
Predicted: 0 | Actual: 0
Predicted: 4 | Actual: 4

AI Model successfully trained and saved!
```